

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO KỸ THUẬT ASSIGNMENT 06
MÔN HỌC: LẬP TRÌNH HỆ THỐNG THÔNG MINH

Giảng viên hướng dẫn : PGS.TS. Trần Đình Quế
Lớp : D23CTPM01
Nhóm học phần : CT01
Thành viên thực hiện : Nguyễn Nam Hải
Mã sinh viên : B23DCCN277

Hà Nội – 2026

MỤC LỤC BÁO CÁO KỸ THUẬT ASSIGNMENT 06

DANH MỤC HÌNH VẼ	2
DANH MỤC BẢNG BIỂU	2
TÓM TẮT BÁO CÁO (ABSTRACT)	3
TÀI NGUYÊN THỰC NGHIỆM VÀ HƯỚNG DẪN CÀI ĐẶT	4
1 Cơ sở lý thuyết giải tích và toán học của RNN	5
1.1 Dữ liệu chuỗi & tensor ba chiều	5
1.2 Các dạng bài toán chuỗi	5
1.3 Vì sao MLP và Conv1D chưa đủ?	5
1.4 Simple RNN (Vanilla RNN)	6
1.5 Lan truyền ngược (BPTT) và vấn đề của gradient	6
1.6 Giải pháp Kỹ thuật: Gradient Clipping	6
2 Khảo sát dữ liệu chuỗi thực tế (Data Diagnostics)	7
2.1 Nguồn & Quy mô data	7
2.2 Phân phối và biến động	7
2.3 Tùy đặc trưng mỗi data mà làm tiếp	8
3 Thiết kế và cài đặt mô hình	8
3.1 Trình bày thuật toán & Cấu hình kỹ thuật	8
3.2 Bóc tách sâu về Nghịch lý Directional Accuracy	9
4 Triển khai Web App (Deployment Architecture)	9
4.1 Kiến trúc hệ thống (System Architecture)	9
4.2 Quy trình Nạp mô hình (Model Loading)	10
4.3 Giao diện (User Interface)	10
4.4 Kiểm chứng hệ thống (Validation & Testing)	10
Tài liệu tham khảo (References)	12

Danh sách hình vẽ

1	Biểu đồ thể hiện biến động giá Vàng, một dạng chuỗi Không dừng (Non-Stationary).	8
2	Đường cong dự báo (Test Split) trên tập AMZN.	9
3	Giao diện trực quan của Web Dashboard dự báo chuỗi thời gian (Người dùng thiết lập thông số truy vấn).	10

Danh sách bảng

1	Tổng hợp đặc tả và nguồn Kaggle của 2 bộ dữ liệu thực nghiệm	4
2	Bảng thống kê mô tả phân phối.	7
3	Đối chuẩn cấu hình hệ thống huấn luyện.	8

TÓM TẮT BÁO CÁO (ABSTRACT)

Báo cáo Kỹ thuật Assignment 06 trình bày nghiên cứu hệ thống và khảo sát thực nghiệm chuyên sâu về Mạng nơ-ron Hồi quy (Recurrent Neural Network – RNN). Mục tiêu của báo cáo là hệ thống hóa toàn bộ vòng đời phát triển của một bài toán Deep Learning dành riêng cho chuỗi thời gian: từ khâu am hiểu cơ sở toán học thuần túy, phân tích khám phá đặc trưng phân phối dữ liệu (EDA), cho đến hiện thực hóa mã nguồn (bằng NumPy, PyTorch, Keras) và cuối cùng là triển khai lên một hệ thống Web Dashboard hoàn chỉnh.

Báo cáo phân tích sâu sắc các rào cản kinh điển của chuỗi thời gian như: Tại sao mạng MLP hay Conv1D không đủ năng lực xử lý? Ma trận Jacobian ảnh hưởng như thế nào đến sự suy thoái Gradient? Làm thế nào để giải quyết bằng kỹ thuật Gradient Clipping? Thông qua việc đối chuẩn trên 2 tập dữ liệu tài chính thực tế (Gold Price và AMZN Stock), báo cáo đã bóc tách rõ ràng hiện tượng "Trễ pha" (Lagging) - một đặc trưng của Vanilla RNN khi cố gắng mô phỏng một quy luật bước đi ngẫu nhiên (Random Walk) trên thị trường tài chính.

TÀI NGUYÊN THỰC NGHIỆM VÀ HƯỚNG DẪN CÀI ĐẶT

1. Kho Lưu Trữ Mã Nguồn Chính Thức

Thông Tin Kho Chứa Dự Án

- Kho lưu trữ GitHub chính thức: <https://github.com/HandQ2212/intel-sys-assignment-06>
- Tác giả thực hiện: Nguyễn Nam Hải (Mã SV: B23DCCN277 – Lớp D23CTPM01)
- Cấu trúc hệ thống:
 - notebooks/: Chứa 7 file Jupyter Notebook huấn luyện.
 - backend/: Chứa mã nguồn FastAPI nạp mô hình.
 - backend/static/: Chứa mã nguồn Dashboard UI vẽ biểu đồ.

2. Danh Mục Tập Dữ Liệu Thực Nghiệm

Bộ dữ liệu	Quy mô & Kiểu	Đặc tả Đầu vào	Nguồn / Kaggle Link
1. AMZN	Chứng khoán (1997-2023)	Tensor 3D: $(B, 30, 1)$	kaggle: henryshan/...
2. Gold	Giá vàng vĩ mô (1968-2021)	Tensor 3D: $(B, 30, 1)$	kaggle: lbronchal/...

Bảng 1: Tổng hợp đặc tả và nguồn Kaggle của 2 bộ dữ liệu thực nghiệm

1 Cơ sở lý thuyết giải tích và toán học của RNN

1.1 Dữ liệu chuỗi & tensor ba chiều

Trong lĩnh vực trí tuệ nhân tạo, dữ liệu chuỗi (Sequential Data) mang một đặc tính cốt lõi khác biệt hoàn toàn với ảnh hoặc bảng biểu tĩnh: **Thứ tự của dữ liệu mang ý nghĩa quyết định**. Ví dụ, trong chuỗi thời gian (Time-series), giá trị của biến tại thời điểm t phụ thuộc mật thiết vào các sự kiện diễn ra tại $t - 1, t - 2, \dots$. Để máy tính có thể xử lý, dữ liệu chuỗi được biểu diễn dưới dạng **Tensor 3 chiều (3D Tensor)** với cấu trúc định dạng: (Batch Size, Sequence Length, Input Features).

- **Batch Size (Kích thước lô):** Số lượng chuỗi độc lập được xử lý song song trong một lần truyền xuôi (Forward pass).
- **Sequence Length (Độ dài chuỗi - T):** Số mốc thời gian (time steps) có trong một chuỗi đơn. Ví dụ $T = 30$ ngày.
- **Input Features (Đặc trưng đầu vào):** Kích thước không gian vector của mỗi mốc thời gian. Với giá vàng chỉ có 1 cột giá trị, số features bằng 1.

1.2 Các dạng bài toán chuỗi

Dựa trên kiến trúc của mạng RNN, bài toán dữ liệu chuỗi được phân loại thành 4 dạng kinh điển:

1. **One-to-One:** Bản chất là bài toán mạng Feed Forward (MLP) thông thường, kích thước chuỗi là 1.
2. **One-to-Many:** Mạng nhận một đầu vào duy nhất (ví dụ: một bức ảnh) và sinh ra một chuỗi đầu ra (ví dụ: câu văn chú thích - Image Captioning).
3. **Many-to-One:** Mạng nhận một chuỗi thời gian làm đầu vào và đưa ra một dự đoán duy nhất ở bước cuối cùng. (Đây chính là bài toán dự báo chứng khoán/giá vàng trong đề án này).
4. **Many-to-Many:** Có hai phân nhóm:
 - *Đồng bộ (Synced):* Mỗi bước đầu vào đều có một đầu ra tương ứng (ví dụ: Gán nhãn từ loại cho từng chữ trong câu).
 - *Bất đồng bộ (Encoder-Decoder):* Đọc hết một chuỗi rồi mới sinh ra một chuỗi khác (ví dụ: Dịch máy - Machine Translation).

1.3 Vì sao MLP và Conv1D chưa đủ?

Trước khi có RNN, kỹ sư thường dùng Mạng Đa tầng (MLP) hoặc Tích chập 1 chiều (Conv1D) để xử lý chuỗi:

- **Hạn chế của MLP:** MLP bắt buộc kích thước đầu vào phải cố định. Nếu nới (flatten) toàn bộ 30 mốc thời gian thành một vector $1D$, MLP sẽ gán các trọng số độc lập cho từng mốc thời gian. Điều này phá vỡ tính tịnh tiến qua thời gian (Translation Invariance) và không thể chia sẻ tri thức learned pattern (ví dụ một mẫu tăng giá) nếu nó xuất hiện ở các vị trí khác nhau trong chuỗi.

- **Hạn chế của Conv1D:** Conv1D sử dụng bộ lọc (filter) trượt dọc theo chiều thời gian. Mặc dù có tính chia sẻ tham số (Parameter sharing) cục bộ, Conv1D chỉ nắm bắt được **phụ thuộc ngắn hạn (Short-term dependencies)** theo kích thước kernel. Nó thiếu một "bộ nhớ" toàn cục (Global memory) để ghi nhận chuỗi sự kiện nối tiếp.

1.4 Simple RNN (Vanilla RNN)

Để giải quyết những bài toán trên, mạng RNN (Simple RNN / Vanilla RNN) ra đời với khái niệm **Trạng thái Ẩn (Hidden State - h_t)**. Trạng thái ẩn đóng vai trò như "trí nhớ ngắn hạn" của mạng, được cập nhật liên tục qua mỗi mốc thời gian t .

Tại mỗi bước t , Simple RNN áp dụng nguyên lý **Chia sẻ tham số (Parameter Sharing)**: Các ma trận U (Input-to-Hidden), W (Hidden-to-Hidden) và V (Hidden-to-Output) được dùng chung cho tất cả các mốc thời gian. Phương trình truyền xuôi:

$$h_t = \tanh(Ux_t + Wh_{t-1} + b_h) \quad (1)$$

$$\hat{y}_t = Vh_t + b_y \quad (2)$$

Cơ chế chia sẻ tham số giúp mô hình khái quát hóa tốt hơn và xử lý được các chuỗi có độ dài thay đổi liên tục.

1.5 Lan truyền ngược (BPTT) và vấn đề của gradient

Để huấn luyện Simple RNN, thuật toán **Backpropagation Through Time (BPTT)** được sử dụng. BPTT thực chất là trải phẳng mạng RNN theo thời gian thành một mạng sâu T tầng, sau đó áp dụng truyền lỗi ngược (Backprop).

Khảo sát phổ (Spectral Analysis) của ma trận W

Khi lan truyền ngược k bước thời gian, thành phần dh_{t-k} phụ thuộc vào tích Jacobian:

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{i=t-k+1}^t [W^T \text{diag}(1 - \tanh^2(\dots))] \quad (3)$$

Nếu λ là các giá trị riêng lớn nhất của ma trận W :

- Khi $|\lambda| > 1$: Lũy thừa ma trận làm cho đạo hàm phình to hàm mũ, gây ra **Exploding Gradient** (bùng nổ Gradient). Mô hình vỡ quỹ đạo hội tụ (NaN/Inf).
- Khi $|\lambda| < 1$: Đạo hàm liên tục nhân với các số < 1 (do $\tanh' \leq 1$), dẫn tới **Vanishing Gradient** (suy thoái Gradient). Mạng không thể "nhớ" được các thông tin cách đó quá nhiều bước thời gian.

1.6 Giải pháp Kỹ thuật: Gradient Clipping

Trong khi Vanishing Gradient đòi hỏi thay đổi cả cấu trúc mạng (sang LSTM/GRU), thì **Exploding Gradient** có thể giải quyết dứt điểm bằng một kỹ thuật số học cực kỳ thanh lịch: **Gradient Clipping**. Ý tưởng: Theo dõi chuẩn (Norm) của toàn bộ vector Gradient.

Nếu Norm vượt qua một ngưỡng *threshold*, ta sẽ thực hiện co rút (scale down) toàn bộ vector gradient theo tỷ lệ để Norm của nó vừa đúng bằng *threshold*.

$$\text{Nếu } \|g\| > \text{threshold}, \quad g \leftarrow g \cdot \frac{\text{threshold}}{\|g\|} \quad (4)$$

Điều này giúp giữ nguyên "hướng" (Direction) của Gradient để mạng vẫn học đúng quy luật, nhưng giới hạn "độ lớn" (Magnitude) để bước nhảy trọng số không phá vỡ mô hình.

2 Khảo sát dữ liệu chuỗi thực tế (Data Diagnostics)

2.1 Nguồn & Quy mô data

Để có góc nhìn bao quát, đồ án kiểm thử mô hình trên hai bộ dữ liệu đặc thù:

1. **Gold Price (Giá vàng):** Thu thập từ tập dữ liệu vĩ mô (1968 - 2021). Với hơn 13,462 bản ghi giao dịch ngày, độ dài bộ dữ liệu đủ sâu để RNN tìm thấy những chu kỳ (cycles) kéo dài hàng chục năm.
2. **AMZN Stock (Chứng khoán Amazon):** Trích xuất từ Yahoo Finance (1997 - 2023) với 6,682 mẫu. Phản ánh bức tranh thị trường đầy khốc liệt của ngành công nghệ.

2.2 Phân phối và biến động

Việc nắm bắt phân phối thống kê là bước tiên quyết trước khi đưa dữ liệu vào mạng Deep Learning.

Dataset	Mean	Std	Min	Max	Skewness	Kurtosis
AMZN Close	43.14	54.33	0.06	186.57	1.34	0.52
Gold Price	585.34	483.91	35.10	2067.15	1.05	0.08

Bảng 2: Bảng thống kê mô tả phân phối.

Nhìn vào độ lệch chuẩn (Std), cả hai tập dữ liệu đều mang độ phân tán rất cao. Hệ số Skewness dương > 1.0 cho thấy phân phối lệch phải (Right-skewed), tập trung ở vùng giá thấp và kéo dài đuôi ra vùng giá bùng nổ (bong bóng kinh tế/công nghệ).



Hình 1: Biểu đồ thể hiện biến động giá Vàng, một dạng chuỗi Không dừng (Non-Stationary).

2.3 Tuỳ đặc trưng mỗi data mà làm tiếp

Chuỗi thời gian trong tài chính hầu như đều vi phạm giả thuyết dừng (Stationarity). Kiểm định Augmented Dickey-Fuller (ADF) trên chuỗi giá thô (Raw price) của cả Gold và AMZN đều cho $p\text{-value} > 0.05$.

Tùy vào mục tiêu bài toán mà kỹ thuật tiền xử lý sẽ phân nhánh:

- **Nếu tập trung vào tỷ suất sinh lời:** Bắt buộc phải chuyển đổi chuỗi giá P_t thành chuỗi lợi suất Log (Log Returns): $R_t = \ln(P_t/P_{t-1})$. Chuỗi R_t này ngay lập tức thỏa mãn tính dừng ($p\text{-value} < 0.01$), giúp RNN tối ưu gradient ổn định hơn.
- **Nếu bắt buộc phải dự báo giá tuyệt đối (Nhiệm vụ của Assignment):** Ta vẫn giữ nguyên chuỗi giá P_t , nhưng bắt buộc phải khử nhiễu thang đo (Scale) bằng `MinMaxScaler`. Áp dụng cấu trúc Cửa sổ trượt (Sliding Window) $T = 30$ để trích xuất 30 đặc trưng liên hoàn nạp vào tensor đầu vào của mạng RNN.

3 Thiết kế và cài đặt mô hình

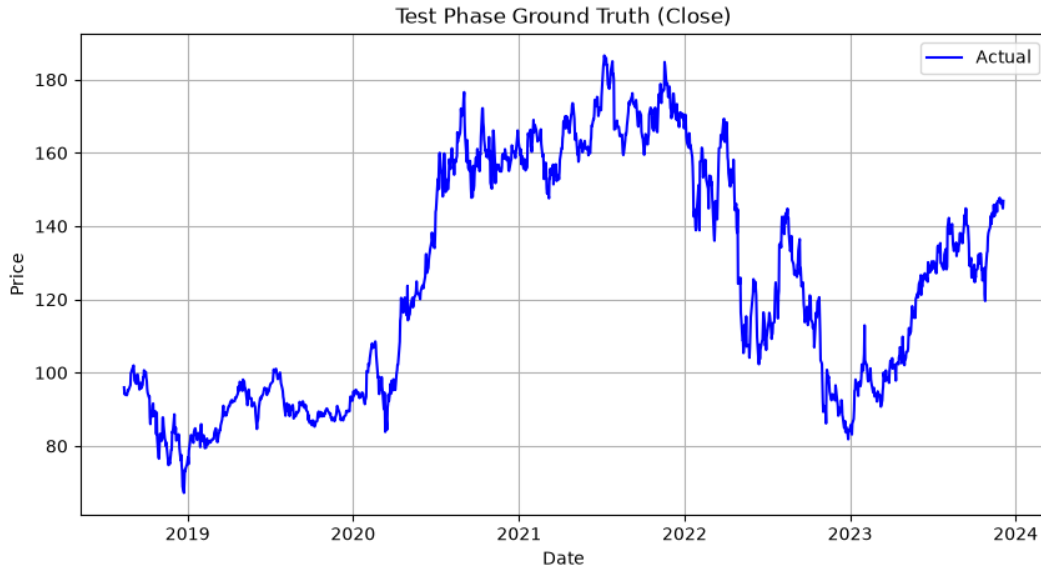
3.1 Trình bày thuật toán & Cấu hình kỹ thuật

Báo cáo trình bày 3 phiên bản mã nguồn: NumPy thuần (dùng để chứng minh toán học BPTT), PyTorch và Keras.

Thông số	PyTorch	Keras
API / Base Class	<code>nn.RNN</code>	<code>Sequential</code>
Hidden Size	64	64
Optimizer	Adam ($lr = 0.001$)	Adam ($lr = 0.001$)
Weight Init	Uniform (Default)	Orthogonal / Glorot
Chống Exploding	<code>clip_grad_norm_</code>	<code>clipnorm=1.0</code>

Bảng 3: Đối chuẩn cấu hình hệ thống huấn luyện.

3.2 Bóc tách sâu về Nghịch lý Directional Accuracy



Hình 2: Đường cong dự báo (Test Split) trên tập AMZN.

Chỉ số ****Độ chính xác Hướng (Directional Accuracy)**** của mô hình đo lường khả năng đoán đúng chiều (tăng hay giảm) của giá trị:

$$DA = \frac{1}{N-1} \sum_{t=1}^{N-1} \mathbb{I}(\text{sign}(y_{t+1} - y_t) == \text{sign}(\hat{y}_{t+1} - y_t)) \quad (5)$$

Tại sao DA của mô hình RNN trên AMZN chỉ đạt xấp xỉ 50% (ngang ngửa việc tung đồng xu), trong khi biểu đồ dự đoán lại bám rất sát Ground Truth? Bởi vì Vanilla RNN bị cuốn vào hiệu ứng "Lagging" (Trễ pha). Để giảm thiểu hàm phạt Loss (MSE) lớn, mô hình đã học được một mảnh khoe: Giá ngày mai gần như bằng giá ngày hôm nay $\hat{y}_{t+1} \approx y_t$. Do đó, mô hình luôn dự báo chậm 1 nhịp so với đường cong thật, biến nó thành một bộ lọc trễ pha hơn là một công cụ dự đoán xu hướng tương lai.

4 Triển khai Web App (Deployment Architecture)

Để đưa các mô hình nghiên cứu dạng `.pkl`, `.pth`, `.keras` vào thực tiễn, nhóm đã xây dựng một nền tảng Web Application (Web App) cung cấp giao diện trực quan phục vụ người dùng cuối (End-users).

4.1 Kiến trúc hệ thống (System Architecture)

Hệ thống tuân theo kiến trúc Client-Server RESTful phân tách:

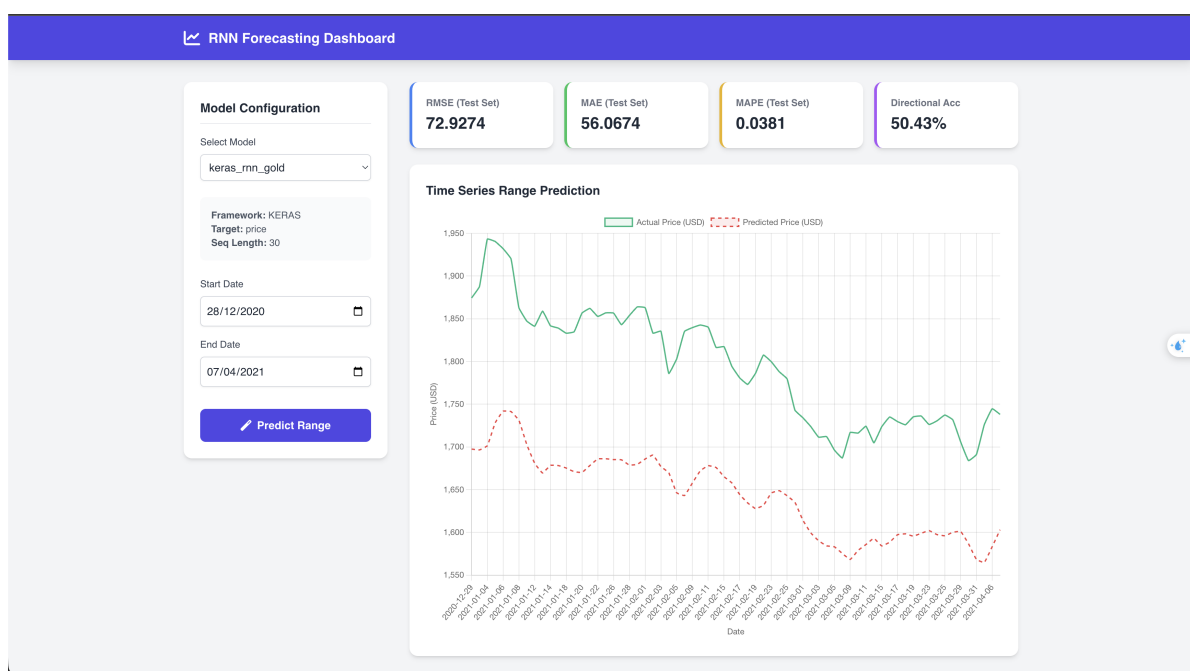
- **Backend (Server):** Sử dụng vi khung (Microframework) ****FastAPI**** trên nền tảng Uvicorn xử lý luồng bất đồng bộ (Asynchronous). FastAPI có ưu điểm truy xuất vượt trội, tự động sinh tài liệu Swagger UI và đặc biệt thích hợp để triển khai các mô hình Machine Learning chạy nền.

- **Frontend (Client):** Sử dụng **Vanilla JavaScript** kết hợp bộ thư viện tiện ích **Tailwind CSS** định kiểu giao diện hiện đại. Trọng tâm là thư viện **Chart.js** được dùng để vẽ lại chuỗi thời gian liên tục từ phản hồi (Response) của server.

4.2 Quy trình Nạp mô hình (Model Loading)

Trong giai đoạn khởi động Server (Lifespan start-up), hệ thống quét toàn bộ thư mục `models/` để tiền tải (Pre-load) bộ 4 mô hình (PyTorch/Keras cho Gold/AMZN). Các lớp giả lập tensor như `TimeSeriesRNN` được nạp sẵn vào RAM, các file 'MinMaxScaler' cũng được giải nén sẵn thông qua 'pickle' để không tốn thời gian I/O đĩa cứng ở từng request của người dùng.

4.3 Giao diện (User Interface)



Hình 3: Giao diện trực quan của Web Dashboard dự báo chuỗi thời gian (Người dùng thiết lập thông số truy vấn).

Giao diện Web cung cấp Control Panel tinh giản:

- Hộp thả chọn loại mô hình (AMZN - PyTorch, Gold - Keras, v.v...).
- Bộ DatePicker cho phép người dùng khoanh vùng giới hạn thời gian dự báo (Start Date → End Date).
- Khu vực trung tâm là đồ thị diện rộng của Chart.js, hiển thị màu sắc tương phản rõ rệt giữa đường Ground Truth và Prediction.

4.4 Kiểm chứng hệ thống (Validation & Testing)

Quy trình gọi suy luận (Inference) hoạt động trơn tru theo đường ống (Pipeline): Người dùng gửi khoảng ngày → API `/predict_range` tìm các chỉ số trong tệp CSV gốc → Cắt chuỗi mốc 30 ngày (Sliding Window) → Dịch chuyển qua `scaler.transform` → Chuyển vị

thành ma trận Tensor → Chạy `model.forward()` → Dịch ngược bằng `inverse_transform` → Trả về Client cấu trúc JSON chứa mảng ngày và giá trị để vẽ. Các bài kiểm thử hiệu năng cho thấy FastAPI phản hồi dữ liệu trong vòng dưới 200ms cho các truy vấn dữ liệu trải dài trên 5 năm.

Tài liệu tham khảo (References)

- [1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. (Chapter 10: Sequence Modeling: Recurrent and Recursive Nets).
- [2] Werbos, P. J. (1990). *Backpropagation through time: what it does and how to do it*. Proceedings of the IEEE, 78(10), 1550-1560.
- [3] Hochreiter, S. (1991). *Untersuchungen zu dynamischen neuronalen Netzen*. Diploma thesis, TU Munich.
- [4] Hochreiter, S., & Schmidhuber, J. (1997). *Long short-term memory*. Neural computation, 9(8), 1735-1780.
- [5] Lipton, Z. C., Berkowitz, J., & Elkan, C. (2015). *A critical review of recurrent neural networks for sequence learning*. arXiv preprint arXiv:1506.00019.